

ESGML (Everlasting Summer Git Mods Loader) v4.1

Полное руководство по архитектуре, портированию модов и SDK

1. Введение и назначение

ESGML (Everlasting Summer Git Mods Loader) — это открытый загрузчик и менеджер модов для визуальной новеллы «Бесконечное Лето», позволяющий скачивать, запускать и обновлять модификации напрямую из Git-репозитория и облачных хранилищ в обход Steam Workshop.

Загрузчик решает ключевую проблему сообщества: **сохранение удалённых, заблокированных или неопубликованных в Мастерской Steam модификаций**.

2. Архитектура загрузчика ESGML v4.1

2.1. Порядок инициализации (Init Order)

В Ren'Py порядок выполнения блоков `init` критически важен. В ESGML выстроена строгая иерархия:

1. `init -999 python: ( renpy_mod_downloader.rpy )`
2. Определение путей к директориям модов.
3. Первоочередной поиск папки Workshop (`workshop/content/331470/1515489831`).
4. Fallback на `game/mods/esgml` при автономном запуске.
5. Инициализация глобальной переменной `git_destination`, предотвращающая ошибки `NameError` при раннем старте скачанных модов.
6. `init -990 python: ( esgml_main_menu.rpy , git_deleter_v2.rpy )`
7. Инициализация постоянных настроек (`persistent.esgml_main_menu_overlay` , `persistent.git_mod_deleting`).
8. `init 1-10 python:`

9. Регистрация модов через `git_easyrepo()` .
10. Формирование списков очереди (`git_queue` , `git_del_queue`).
11. `init 999 python:`
12. Монтирование системных архивов интерфейса (`res_git.rpa` , `git_screens.rpa`) в `config.archives` .
13. `init 1000 python:`
14. Подключение хуков интерфейса (`config.interact_callbacks`) для быстрого доступа из главного меню.

2.2. Динамическое монтирование архивов (`rpa_check_append`)

В отличие от стандартных модов, которые распаковываются в папку игры, ESGML использует компактное монтирование `.rpa` на лету:

```
init 1 python:
    rpa_check_append('git_<alias>_res.rpa', 'git_<alias>_res')
    rpa_check_varinst('<alias>', u'<Название мода> ESGML', 'git_<alias>_res.rpa')
```

- Если файл архива найден на диске, архив регистрируется в `renpy.config.archives` .
- Если файл отсутствует (мод ещё не скачан или удалён), мод не появляется в основном меню игры, предотвращая вылеты из-за отсутствующих ассетов.

2.3. Решение проблемы блокировки файлов Windows (File Locking)

На Windows запущенный процесс игры удерживает файловые дескрипторы открытых `.rpa` архивов (`ERROR_SHARING_VIOLATION`), что делает невозможным их мгновенное удаление с диска.

Реализованный механизм:

1. При удалении мода он немедленно отключается из `renpy.config.archives` и удаляется из `persistent.git_mod_installed` .
2. Если `os.remove()` возвращает ошибку блокировки доступа, путь к файлу помещается в очередь отложенного удаления: `persistent.git_mod_deleting` .
3. При следующем запуске игры модуль `git_postdeleter.rpy` удаляет освобождённые файлы без искусственных задержек и «сна».

2.4. Очереди загрузки и удаления

- **Очередь загрузки (`git_queue`)**: Позволяет выбрать несколько модов из каталога и скачать их пакетом с отображением общего прогресса.
- **Очередь удаления (`git_del_queue`)**: Позволяет массово очистить память от ненужных модов.
- **Флаг `git_qu_lock`** : Во время пакетной обработки подавляет промежуточные всплывающие окна. По завершении очереди выводится единое модальное окно с предложением перезагрузки.

2.5. Оверлей быстрого доступа в главном меню

Модуль `esgml_main_menu.rpy` выводит полупрозрачную панель прямо в левом верхнем углу главного меню:

- **[Моды]** — мгновенный переход к списку модов игры (`ShowMenu('mods')`).
- **[ESGML]** — прямой запуск загрузчика.
- Оверлей автоматически скрывается во время чтения новеллы и может быть отключён через «Настройки и отладка».

3. Анатомия адаптированного мода

Каждый мод под загрузчик ESGML состоит из четырёх элементов:

1. `git_res.rpa` — архив ресурсов мода (музыка, звуки, спрайты, фоны, шрифты).
2. `git_base.rpyc` — скомпилированный скрипт мода, содержащий:
3. Вызовы `rpa_check_append` и `rpa_check_varinst`.
4. Сценарий (`label : ...`).
5. **Превью-скриншоты (`(1).png` , `(2).png` , `(3).png`)** — изображения 480x270 для карточки мода.
6. **Запись в репозитории (`git_default_repo.rpy`)**:

```
init 5 python:
    git_easyrepo(
        'samantha',
        ("https://github.com/user/repo/releases/download/v1.0/git_samantha_base.r
        "https://github.com/user/repo/releases/download/v1.0/git_samantha_res.rp
        'Саманта',
```

```
u'''Описание мода, сюжетная завязка и авторы.'''
)
```

4. Пошаговое ручное портирование мода

Если вы хотите портировать мод вручную без SDK:

Шаг 1: Подготовка файлов

1. Найдите папку мода в Steam Workshop:

```
steamapps/workshop/content/331470//mods// .
```

2. Разделите файлы на:

3. **Скрипты:** `.rpy` и `.rpyc` .

4. **Ресурсы:** папки `image/` , `sound/` , `music/` , `sprites/` , `bg/` , `fonts/` .

Шаг 2: Упаковка ресурсов в RPA

Упакуйте ресурсы в архив `git_res.rpa` через официальный инструмент Ren'Py или `rpa-pack` :

```
python sdk/esgml_cli.py rpa-pack path/to/assets git_mymod_res.rpa
```

Шаг 3: Добавление заголовка в скрипт

В начале главного сценария `.rpy` добавьте блок:

```
init 1 python:
    rpa_check_append('git_mymod_res.rpa', 'git_mymod_res')
    rpa_check_varinst('mymod', u'Название мода ESGML', 'git_mymod_res.rpa')
```

Закомментируйте старую строку регистрации вида `mods["..."] = ...` .

Шаг 4: Компиляция в .rpyc

Скомпилируйте скрипт с помощью Ren'Py в `git_mymod_base.rpyc` .

Шаг 5: Создание превью

Подготовьте 3 изображения в формате PNG (480x270):

- `mymod (1).png`
- `mymod (2).png`
- `mymod (3).png`

5. Автоматическое портирование через ESGML SDK

ESGML SDK автоматизирует весь процесс в один клик.

5.1. Использование GUI (Графический интерфейс)

Запустите графическое приложение:

```
python sdk/esgml_gui.py
```

(Или запустите скомпилированный `ESGML_Tool_GUI.exe`)

1. Перейдите на вкладку « **Портирование мода**».
2. Нажмите «**Обзор...**» и выберите папку мода из Workshop или распакованного архива.
3. SDK автоматически определит:
4. Название мода.
5. Стартовый лейбл.
6. Алиас на латинице.
7. Превью-изображение.
8. Нажмите « **Адаптировать и собрать под ESGML**».
9. В папке `output/` будут созданы:
10. `git__res.rpa`
11. `git__base.rpyc`
12. 3 готовых скриншота
13. `repo_entry.py` с готовым кодом для репозитория.

5.2. Использование CLI (Консольный режим)

Команда автоматической адаптации:

```
python sdk/esgml_cli.py adapt "E:/workshop/content/331470/1081934648" -
```

6. Встроенные утилиты RPA и RPYC

SDK содержит полнофункциональные инструменты на чистом Python без внешних библиотек:

6.1. Работа с RPA архивами

■ Упаковка папки в RPA v3:

```
```bash
```

```
python sdk/esgml_cli.py rpa-pack my_folder my_archive.rpa --ver 3
```

```
```
```

■ Распаковка архива:

```
```bash
```

```
python sdk/esgml_cli.py rpa-unpack my_archive.rpa
extracted_folder/
```

```
```
```

■ Просмотр содержимого:

```
```bash
```

```
python sdk/esgml_cli.py rpa-list my_archive.rpa
```

```
```
```

6.2. Работа со скриптами RPYC

■ Анализ структуры (поиск лейблов и регистрации мода):

```
```bash
```

```
python sdk/esgml_cli.py rpyc-info script.rpyc
```

```
```
```

■ Декомпиляция байткода:

```
```bash
```

```
python sdk/esgml_cli.py rpyc-decompile script.rpyc --out script.rpy
```

```
```
```

7. Массовый анализ архива Workshop и поиск удалённых модов

В SDK встроен скоростной анализатор архивов Мастерской Steam через официальный **Steam Web API** с пакетированием до 100 модов в одном HTTP-запросе:

```
python sdk/esgml_cli.py batch-scan "E:/ANALGAYPORNSTEAM/steamapps/works
```

Возможности анализатора:

- За несколько секунд проверяет весь архив (1900+ модов).
- Фильтрует пустые и повреждённые папки.
- Находит моды со статусом `k_EResultFileNotFound` (9) — **моды, навсегда удалённые из Steam.**
- Формирует отчёт в форматах Markdown и CSV со списком кандидатов для добавления в ESGML.

8. Бесплатный хостинг файлов модов

Поскольку проект некоммерческий, используйте проверенные бесплатные платформы с прямыми HTTP-ссылками:

Вариант 1: GitHub Releases (Рекомендуемый)

1. Создайте публичный репозиторий на GitHub (например, `esgml_mods`).
2. Создайте релиз (Release) с тегом, например `v1.0` .
3. Прикрепите файлы `git_base.rpyc` и `git_res.rpa` в Assets релиза (лимит: до 2 ГБ на файл).
4. Прямая ссылка для скачивания строится по шаблону:

...

<https://github.com///releases/download//>

...

Вариант 2: GitLab Raw

- GitLab даёт до 10 ГБ на репозиторий с бесплатным трафиком.
- Прямая ссылка строится через `raw` :

...

<https://gitlab.com///raw/master/>

...

Вариант 3: Cloudflare R2

- 10 ГБ бесплатного постоянного хранилища.
- **0 руб. за исходящий трафик** (в отличие от AWS S3).
- Поддержка пользовательского домена или бесплатного домена `r2.dev` .

Вариант 4: Internet Archive (archive.org)

- Идеально для постоянного сохранения истории и цифрового наследия новеллы.
- Неограниченное бесплатное хранение с прямым доступом по HTTP.

9. Сборка автономного исполняемого файла (.exe)

Чтобы собрать портативный `.exe` без зависимостей:

```
python sdk/build_exe.py
```

Собранные бинарники будут находиться в папке `dist_bin/`:

- `dist_bin/ESGML_Tool_GUI/ESGML_Tool_GUI.exe` — графическая версия.
- `dist_bin/esgml_cli.exe` — консольная утилита.

10. Добавление собственных репозиториев (папка `repos/`)

Чтобы добавить свои моды в ESGML, **не нужно изменять оригинальный файл** `git_default_repo.rpy` ! При обновлении загрузчика ваши добавленные моды никогда не перезапишутся.

`git_default_repo.rpy` является **базовым встроенным каталогом и образцом**.

Рекомендуемый формат: Ren'Py скрипт (`.rpy`)

> [!TIP]

> **Почему .rpy рекомендуется как основной формат:**

> Файлы `.rpy` компилируются движком Ren'Py нативно, обеспечивают максимальную производительность, поддерживают полную работу с юникодом (русский язык, спецсимволы) и исключают ошибки парсинга.

Создайте файл в папке `repos/` (например, `repos/my_mods.rpy`):

```
# -*- coding: utf-8 -*-
init 5 python:
    git_easyrepo(
        'my_cool_mod',
        ("https://archive.org/download/item_id/git_mymod_base.rpyc",
         "https://archive.org/download/item_id/git_mymod_res.rpa"),
        'Мой крутой мод',
        u'''Описание сюжета, авторы и особенности модификации.''' ,
        scr1="https://archive.org/download/item_id/screen1.png",
        scr2="https://archive.org/download/item_id/screen2.png",
        scr3="https://archive.org/download/item_id/screen3.png"
    )
```

Альтернативный формат: JSON манифест (`.json`)

Подходит для автоматической генерации скриптами или если вы не хотите писать код. Создайте файл

`repos/my_mods.json` :

```
[
  {
    "id": "my_cool_mod",
    "name": "Мой крутой мод",
    "desc": "Описание сюжета...",
    "scr1": "https://archive.org/download/item_id/screen1.png",
    "scr2": "https://archive.org/download/item_id/screen2.png",
    "scr3": "https://archive.org/download/item_id/screen3.png",
    "links": [
      "https://archive.org/download/item_id/git_mymod_base.rpyc",
      "https://archive.org/download/item_id/git_mymod_res.rpa"
    ]
  }
]
```

> [!NOTE]

> **Автоматическое кэширование скриншотов:** В

параметрах `scr1` , `scr2` , `scr3` можно указывать как локальные файлы (`git_screens/...`), так и прямые ссылки `https://` . ESGML скачает картинки в фоне при открытии меню и сохранит их в кэш без лагов и падений!

ESGML автоматически сканирует папку `repos/` при запуске игры и при каждом входе в меню, мгновенно добавляя найденные моды в каталог.

11. Адаптация модов с помощью ИИ-агентов (AI Agent Assisted Pipeline)

Адаптация старых модов (особенно удалённых из Steam 5-10 лет назад) вручную может быть трудоёмкой: часто встречаются устаревшие конструкции Ren'Py 6.x, скрытые UTF-8 BOM символы, конфликтующие вызовы `mods[...]` или синтаксические несовместимости.

Использование современного **ИИ-агента для программирования** (Google Antigravity, Claude Code, Cursor, Codex) превращает рутинный процесс портирования в полностью автоматизированный цикл.

Что ИИ-агент делает под капотом:

1. **Поиск и аудит:** Запускает `batch-scan` для поиска удалённых работ в локальном архиве Steam Workshop.
2. **Адаптация структуры:** Запускает `ModAdapter` из SDK, упаковывая спрайты и фоны в единый RPA-архив.
3. **Анализ и исправление Ren'Py синтаксиса:**
4. Удаляет невидимые символы `\ufeff` (UTF-8 BOM), ломающие парсер Ren'Py.
5. Исправляет устаревшие конструкции операторов звука (`play sound xn -> $ renpy.play(xn, 'sound')` или `play sound "..."`).
6. Корректирует отступы и комментирует старые регистрации в меню игры.
7. **Компиляция байткода:** Прогоняет полученный базовый скрипт через компилятор движка игры для получения стабильного `.rpyc`.

8. **Вечное сохранение:** Загружает готовый пакет на Internet Archive через встроенный `upload_archive_org.py`.

1. **Регистрация в ESGML:** Формирует готовый файл `repos/.rpy` с прямыми ссылками и проверяет его работоспособность в загрузчике.

Шаблон промпта для ИИ-агента:

Скопируйте следующий текст и отправьте вашему ИИ-ассистенту:

Привет! Помоги мне адаптировать мод из Steam Workshop для
- Путь к исходникам мода: [укажите путь, например: E:\workshop\mod]\n- Желаемый алиас: [например: my_mod]

Инструкция:

1. Просканируй структуру папки через SDK:
python sdk/esgml_cli.py adapt "<путь>" --alias <алиас>
2. Проверь получившийся git_<алиас>_base.rpy на синтаксические ошибки
3. Залей полученный пакет на Archive.org с помощью:
python sdk/upload_archive_org.py .dist/<алиас> --item <название>
4. Сгенерируй готовый файл репозитория repos/<алиас>.rpy
5. Удали сырые .rpy из .dist/ и проверь, что мод корректно работает

Благодаря такой синергии вы можете спасать и адаптировать десятки удалённых визуальных новелл за считанные минуты!